Heart disease classification

via modified autoencoder

dataset

DESCRIPTION

- Cardiovascular_Disease_Dataset, a heart disease dataset
- Presence of heart disease depending on various factors

VALUES

- 1000 subjects
- 12 features (patient identification, age, gender, chest pain type, resting blood pressure, serum cholesterol, resting electrocardiogram results, maximum heart rate achieved, exercise-induced angina, oldpeak, slope of peak exercise, number of major vessels, classification)

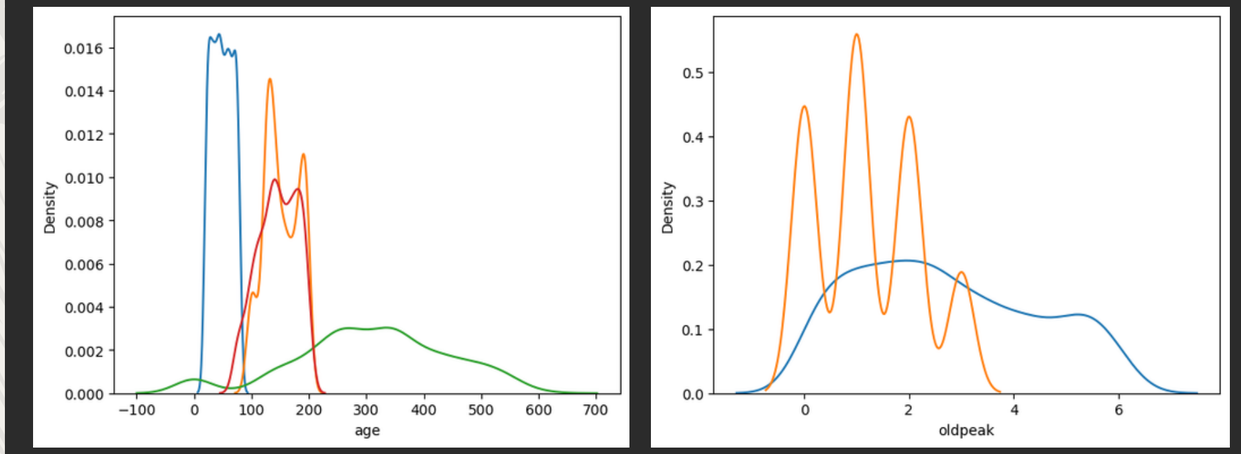
Source of the dataset:

Doppala, Bhanu Prakash; Bhattacharyya, Debnath (2021), “Cardiovascular_Disease_Dataset”, Mendeley Data, V1, doi: 10.17632/dzz48mvjht.1

data preparation

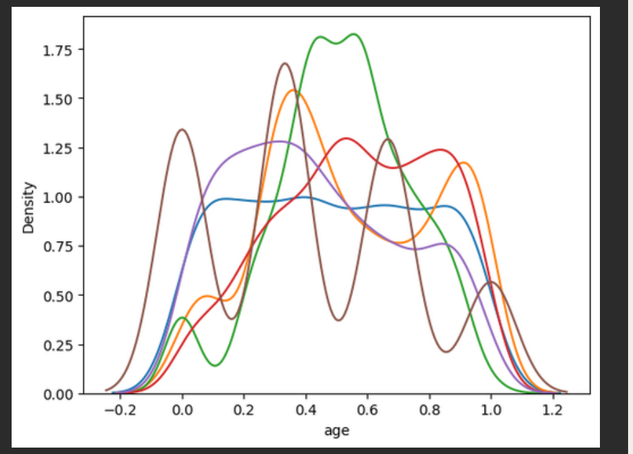
DROPPED COLUMNS

The *patientid* column has been dropped as this is only an identifier for each row. Meanwhile, other binary and nominal features were dropped, namely *gender, chestpain, fastingbloodsugar, exerciseangia, slope,* and *target.*



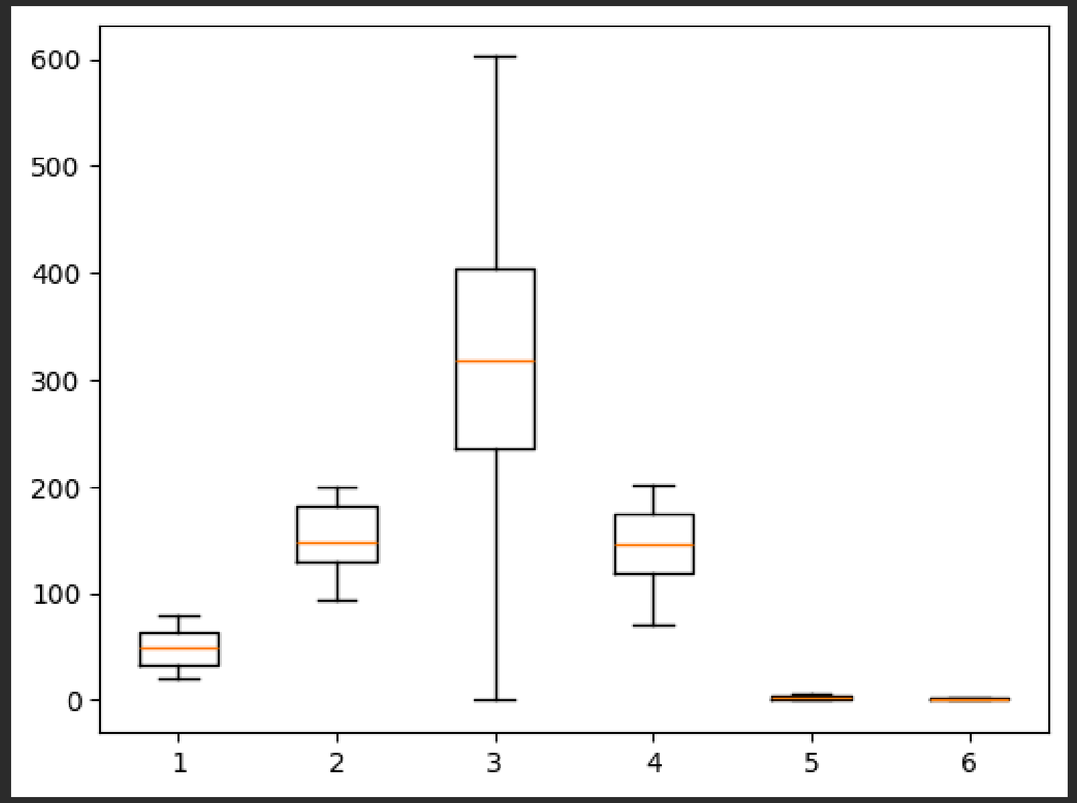
Kernel density estimate plot of data before scaling

MinMaxScaler()



Data plot after MinMax scaling

data preparation



Data plot before MinMax scaling

SCALING DATA

Data was scaled using MinMaxScaler as the data:

- does not follow normal distribution, hence the StandardScaler is not preferable
- does not have outliers, hence the RobustScaler is not chosen

autoencoder modifications

ACTIVATION FUNCTION

- **Rectified linear unit (ReLU)** was retained in the **encoding** aspect of the activation function as it introduces sparsity to the network, which helps to reduce overfitting. (Rallabandi, 2023)
- Meanwhile, the **softmax** activation function was used in **decoding** the compressed data for generalization and classify it into dimensions (Basuki, 2025).

LOSS FUNCTION

- The cross-entropy loss function was used in evaluating the difference between the predicted probability distribution and the true class, which the network uses in assigning higher probability to the correct class during back propagation (devender, 2024).

model structure comparison

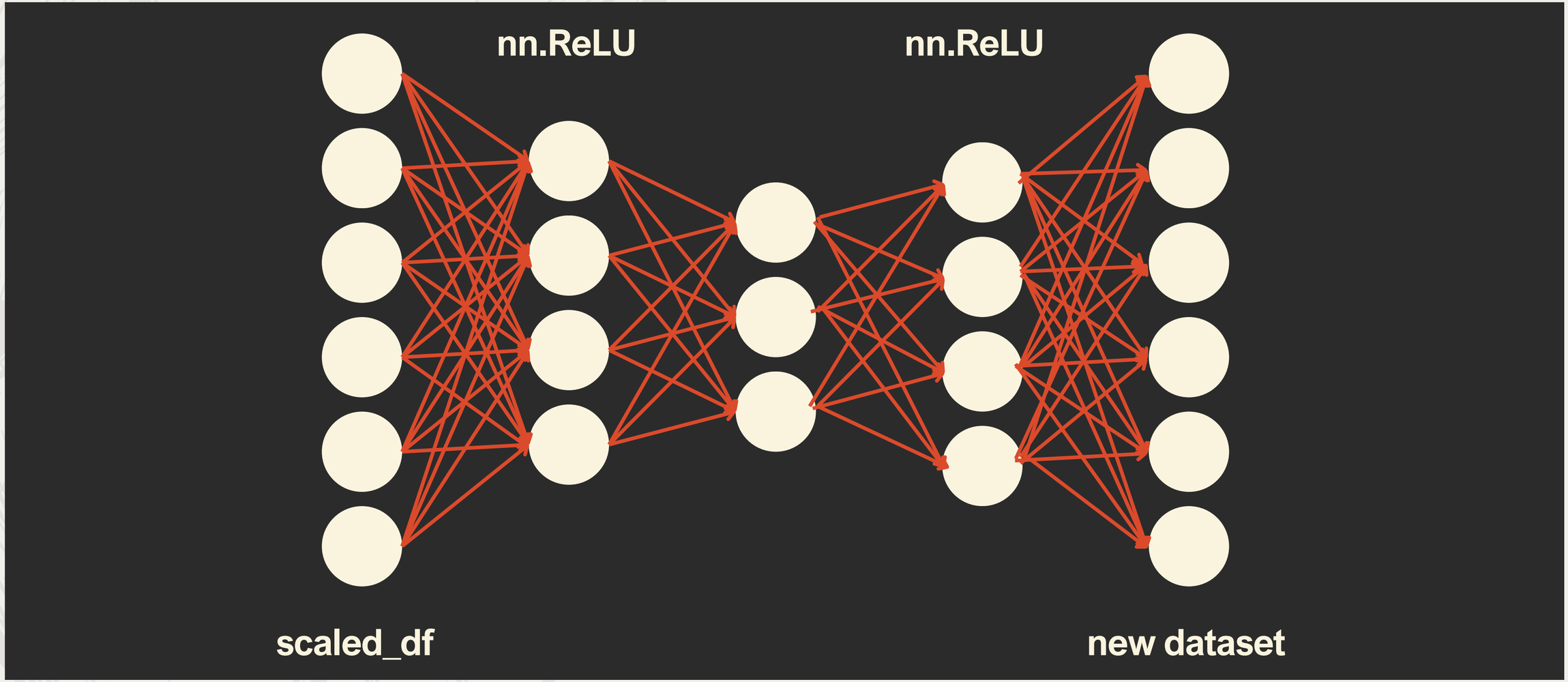
```python
# Encoder: Compresses data
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 4),
            nn.ReLU(),
            nn.Linear(4, latent_dim) # Bottleneck
        )

        # Decoder: Reconstructs data
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 4),
            nn.ReLU(),
            nn.Linear(4, input_dim)
```

VANILLA AUTOENCODER

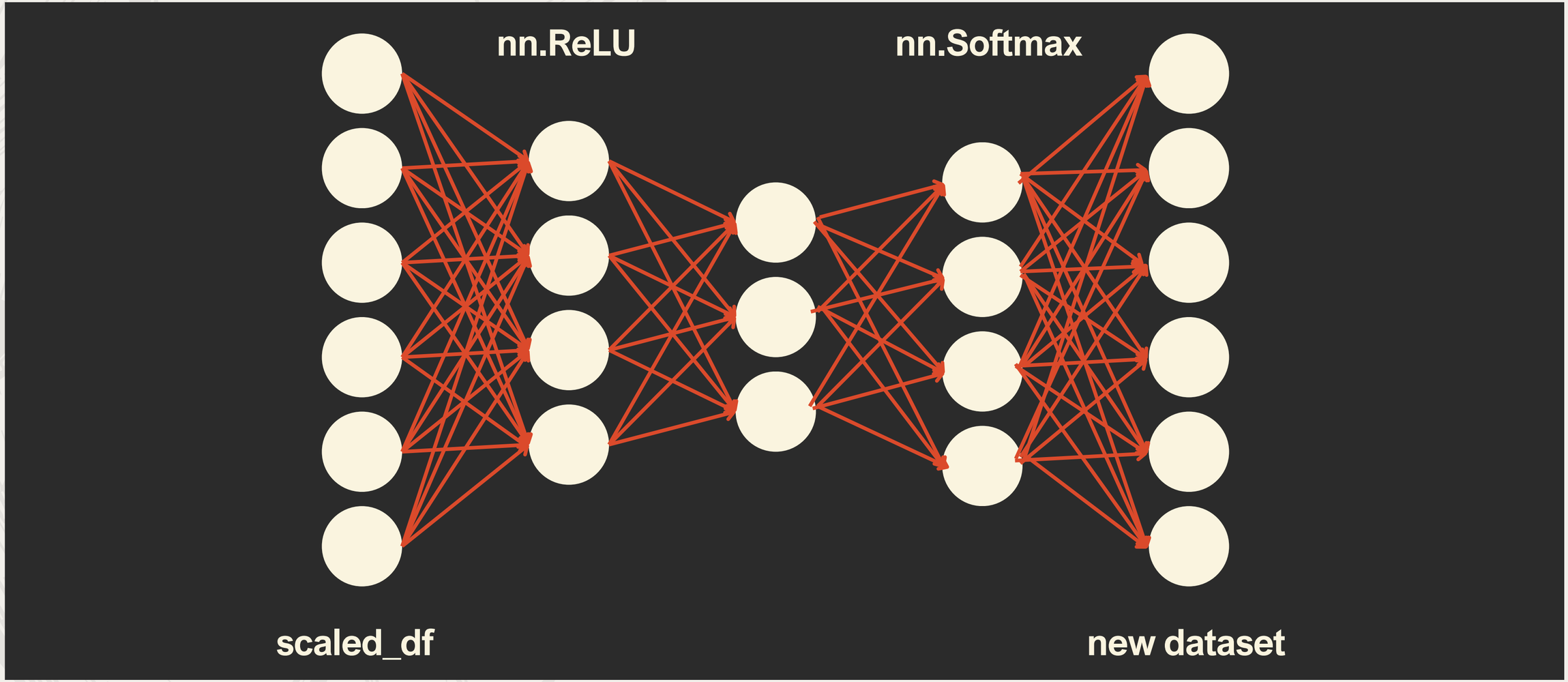
```python
# Encoder: Compresses data
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 4),
            nn.ReLU(),
            nn.Linear(4, latent_dim) # Bottleneck
        )

        # Decoder: Reconstructs data
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 4),
            nn.Softmax(dim=1),
            nn.Linear(4, input_dim)
```

MODIFIED AUTOENCODER

vanilla autoencoder



modified autoencoder

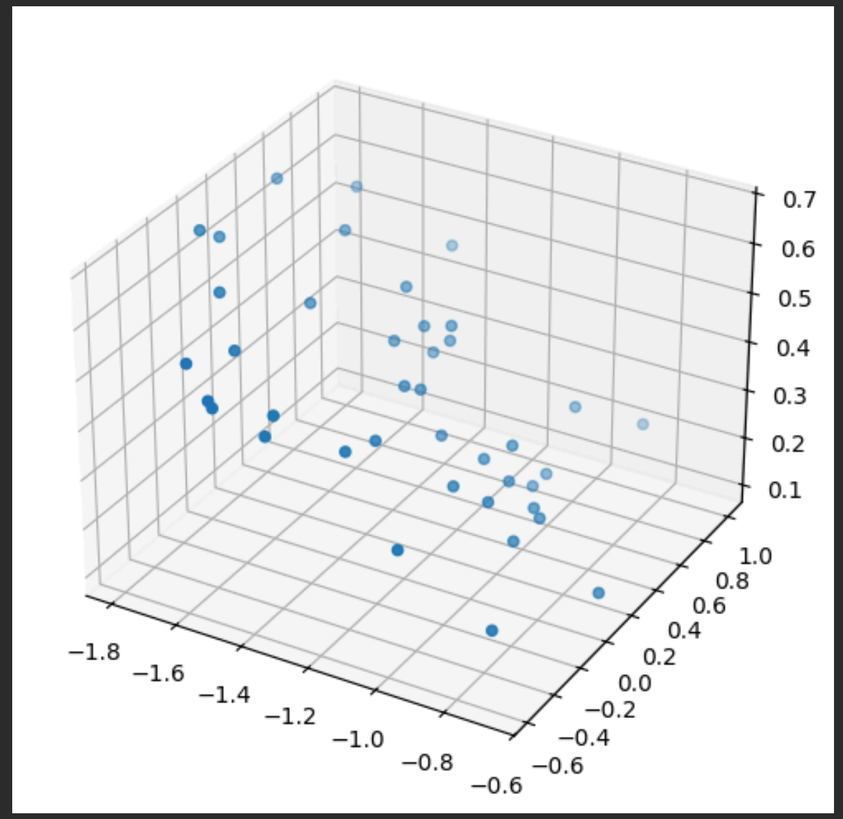


results

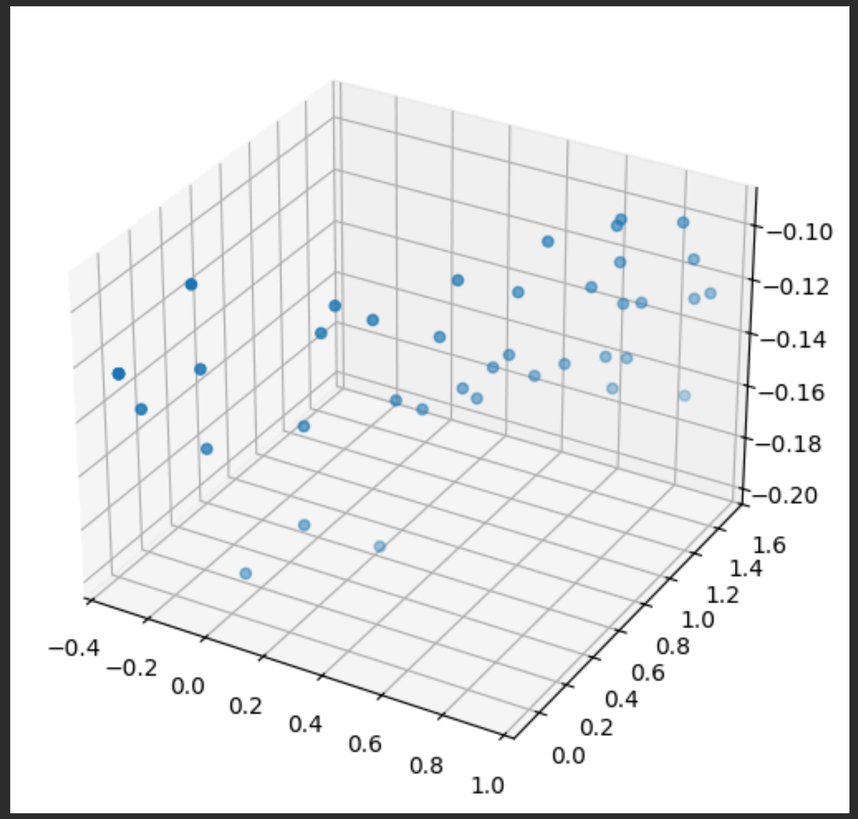
	Original data space	Vanilla autoencoder	Modified autoencoder
K-means	4 - 6	3	2 - 4
From Davies-Bouldin	8	7	3
From Calinski-Harabasz	2	8	8
Selected K	8	3	3

SILHOUETTE SCORES:

0.1546	**0.3733**	**0.5772**



VANILLA AUTOENCODER



MODIFIED AUTOENCODER

results

discussion

MODEL LEARNING

- The modified autoencoder achieves a better silhouette score, i.e. that the model is learning well, due to the usage of another activation function in the output layer, namely softmax. Softmax takes in the logits (numbers) from the compressed data and outputs it as a probability distribution (Nomidi, 2025). Meanwhile, the rectified linear unit helps the model learn non-linear patterns, and is not exactly designed to output probabilities (Amit, 2024; Nomidi, 2025). Hence, the vanilla model, which puts ReLU in decoding the compressed data, achieves a lower silhouette score.

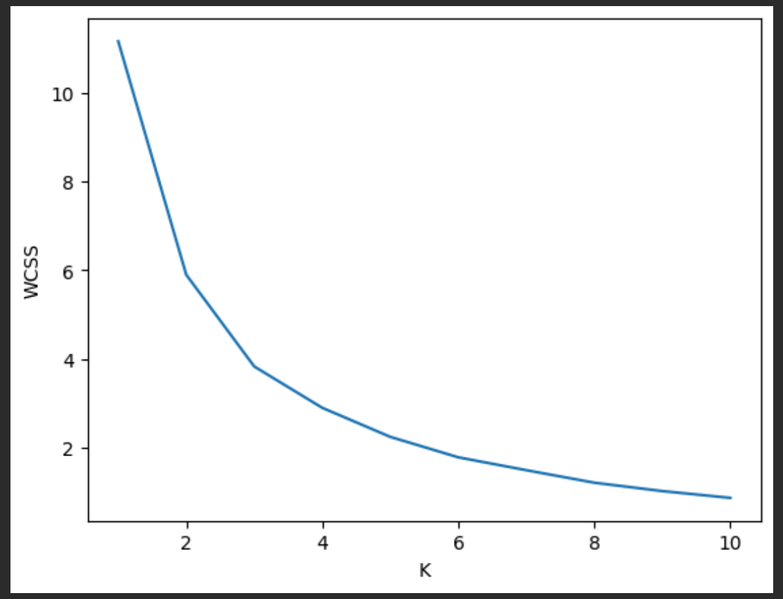
FEATURE REPRESENTATION

- In both autoencoders, feature representation has been reduced from six down to just three. This was controlled in the latent dimension parameter of the autoencoder.

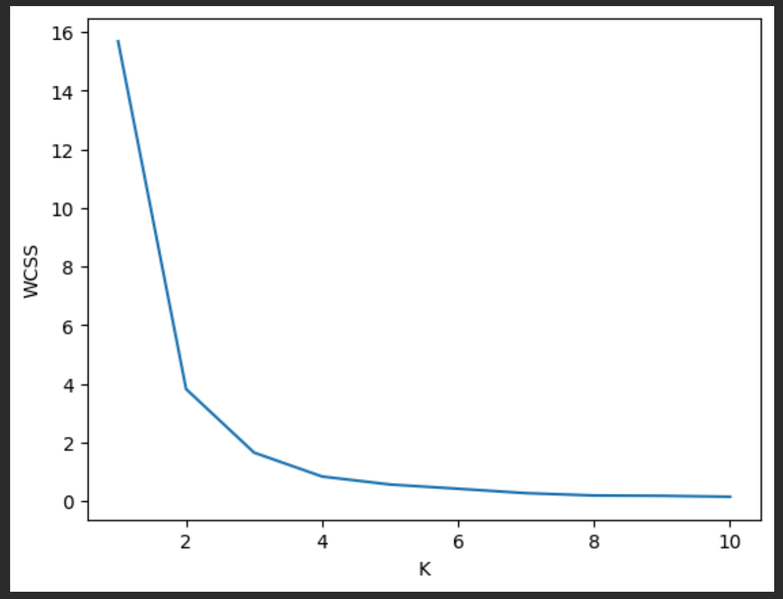
discussion

CONVERGENCE

- For both models, the k-means clustering algorithm was more-or-less able to successfully identify the clusters which fit the compressed data. Notable in the modified autoencoder, however, is the sharper drop in inertia as the number of clusters approached 2.



Vanilla



Modified

discussion

GENERALIZATION

- As the model added softmax as the decoder function, it helped the autoencoder better when it comes to generalization. This was the case as the decoding part of the autoencoder extended the model from its compressed state to a more complex one by outputting a probability distribution over all classes (Thakur, 2022). This is in comparison with ReLU as a decoder, which was earlier established to be not as good in generalizing in comparison to softmax.

CONCLUSION AND RECOMMENDATIONS

- The modified autoencoder was able to demonstrate the ability of softmax to generalize the values better compared to ReLU. This is based on the silhouette score from clustering the dimensionally-reduced data.
- However, more can be determined when it comes to the amount of hidden layers the autoencoder has, as the currently modified one only has one for each encoder and decoder.

Thank you.